

ShipShape

Auditing and Improving a Production TypeScript Codebase

Target Repository: US-Department-of-the-Treasury/ship

<https://github.com/US-Department-of-the-Treasury/ship>

Background

Every software engineer who joins a new company faces the same challenge on day one: a codebase they have never seen, built by people they have never met, using patterns they may not recognize. The ability to orient yourself in an unfamiliar system, assess its health, and improve it with confidence is the defining skill that separates junior engineers from senior ones.

Ship is a project management tool built by the U.S. Department of the Treasury. It combines documentation, issue tracking, and sprint planning into a single application. The codebase is a TypeScript monorepo with a React frontend, Express backend, PostgreSQL database, and real-time collaboration powered by WebSockets and Yjs. It has 73+ Playwright E2E tests, Docker and Terraform deployment configs, and a unified document model where everything (issues, docs, projects, sprints) lives in a single table.

This project does not ask you to build a new feature from scratch. It asks you to do what production engineers actually do: inherit a system, understand it deeply, measure its health, diagnose its weaknesses, and make it better with proof.

Gate: Project completion is required for Austin admission.

Project Overview

Seven-day sprint with two phases:

Checkpoint	Deadline	Focus
Orientation	4 hours after receiving the project	Read the codebase, complete orientation checklist
Audit Report (MVP)	Tuesday, 11:59 PM (36 hours)	Baseline measurements for all 7 categories
Early Submission	Friday, 11:59 PM	Measurable improvements across all 7 categories
Final Submission	Sunday, 11:59 PM	Polish, documentation, presentation

Phase 1: The Audit (36 Hours)

Hard gate. You must submit a written audit report with baseline measurements for all 7 categories below. This report is your diagnostic. It tells us you understand the system before you touch it.

For each category, you must:

1. Describe how you measured it (tools, commands, methodology)
2. Provide concrete baseline numbers
3. Identify the specific weaknesses or opportunities you found
4. Rank the severity or impact of each finding

You do not fix anything during the audit. Diagnosis comes before treatment.

Category 1: Type Safety

What you are measuring: The strength of TypeScript's type system as used in this codebase. This includes explicit **any** types, type assertions (**as**), non-null assertions (!), **@ts-ignore** and **@ts-expect-error** directives, untyped function parameters, and implicit **any** from missing return types.

How to Measure

- Run grep or a static analysis tool to count all type safety violations across the codebase
- Check the **tsconfig.json** for strict mode settings. If strict mode is off, run **tsc --strict --noEmit** and count the errors
- Break down violations by package (web/, api/, shared/) and by violation type
- Identify the 5 most violation-dense files and explain why they are problematic

Audit Deliverable

Metric	Total baseline	api/	web/	shared/
Total any types	99	75	24	0
Total type assertions (as)	1,417	964	453	0
Total non-null assertions (!)	305	280	25	0
Total @ts-ignore / @ts-expect-error	1	0	1	0
Strict mode enabled?	Yes			
Strict mode error count (if disabled)	0			
Top 5 violation-dense files	team.ts : 139 accountability.test.ts : 32	weeks.ts : 132	claudio.ts : 75	projects.ts : 50
Most violation dense files	Almost all violations stem from one root cause: raw SQL with pg returns any rows, and every field access requires a manual cast. A thin typed query layer (even just generic row-mapping functions, not a full ORM) would eliminate a majority of the 1,417 'as' violations and make column renames compile-time errors instead of runtime crashes.			

Improvement Target

Eliminate 25% of type safety violations. Every fix must preserve existing functionality (all tests still pass). Superficial fixes do not count. Replacing **any** with **unknown** without proper type narrowing is not an improvement. Each fix must include correct, meaningful types that reflect the actual data.

Category 2: Bundle Size

What you are measuring: The size of the production frontend bundle. Large bundles slow down initial page load, hurt performance on slow networks, and waste bandwidth. You are looking for oversized dependencies, missing code splitting, unused imports, and opportunities to reduce what the browser has to download.

How to Measure

- Build the production frontend and record the total output size
- Use a bundle visualization tool (e.g., **rollup-plugin-visualizer**, **vite-bundle-analyzer**, or **source-map-explorer**) to generate a treemap of the bundle
- Identify the largest chunks and the largest individual dependencies within them
- Check for unused dependencies: cross-reference **package.json** dependencies against actual imports in the source code
- Evaluate whether code splitting is in use and where lazy loading could reduce initial load **Audit**

Deliverable

Metric	Your Baseline
Total production bundle size	~2,189 KB
Largest chunk	index-C2vAyoQ1.js – 2,073
Number of chunks	261 JS + 1 CSS (~240 are tiny icon SVGs)
Top 3 largest dependencies	1. emoji-picker-react (400 KB) 2. highlight.js (378 KB) 3. Yjs (265 KB)
Unused dependencies identified	@tanstack/query-sync-storage-persister not imported anywhere in src/

Improvement Target

15% reduction in total production bundle size, or implement code splitting that reduces initial page load bundle by 20%. Provide before/after bundle analysis output. Removing functionality to shrink the bundle does not count.

Key observations:

- 95% of JS is in one chunk – almost no code splitting. The **emoji-picker-react** and [highlight.js](#) alone account for 778 KB and could be lazy-loaded since they're only needed when a user opens the emoji picker or views a code block.
- Only tab components are lazy-loaded via `React.lazy()` in **documents-tabs.tsx**. Route-level splitting and heavy dependency splitting are not implemented.
- The ~240 icon SVG chunks are tiny (<1 KB each) – not a size concern but a request count concern if HTTP/2 isn't in use.

Two code-splitting bugs:

1. [upload.ts](#) - dynamic import defeated by static imports. Both *FileAttachment.tsx* and *ImageUpload.tsx* statically import [upload.ts](#), and both are statically imported by *Editor.tsx* so [upload.ts](#) is already in the main chunk – the dynamic import in *SlashCommands* resolves instantly from the existing module, splitting nothing.
2. *FileAttachment.tsx* – Same issue. *Editor.tsx* statically imports *FileAttachment*, so it's already in the main chunk before *SlashCommands* tries to lazy-load it.

To fix these, I need to remove the static imports from *Editor.tsx* and instead register those extensions lazily (load *FileAttachmentExtension* and *ImageUploadExtension* on demand via TipTap's extension API). That's a non-trivial refactor since TipTap extensions are typically registered at editor creation time.

Category 3: API Response Time

What you are measuring: How fast the backend responds under realistic conditions. This is not about testing with an empty database. Seed the database with meaningful volume, then measure.

How to Measure

- Seed the database with realistic data: 500+ documents, 100+ issues, 20+ users, 10+ sprints. Use **pnpm db:seed** or write your own seed script
- Identify the 5 most important API endpoints by tracing the frontend's network requests during common user flows
- Benchmark each endpoint using a load testing tool (**autocannon**, **k6**, **hey**, or similar). Record P50, P95, and P99 response times
- Test under concurrent load: 10, 25, and 50 simultaneous connections
- Identify the slowest endpoints and hypothesize why they are slow

Audit Deliverable

Endpoint	P50	P95	P99
1. GET /api/issues	177ms	196ms	203ms
2. GET /api/weeks	16ms	44ms	47ms
3. GET /api/programs	14ms	40ms	44ms
4. GET /api/documents/:id	14ms	40ms	44ms
5. GET /api/dashboard/my-week	24ms	62ms	68ms

Major Bottlenecks:

1. GET /api/issues (203ms P99) – Slowest API. Root causes:
 - Endpoint returns all issues (document type) with the `content` JSONB field included. Every request serializes 310KB of JSON. At 50 concurrent connections, this is 15MB/s of JSON serialization alone. The `content` field is the full TipTap document body - unnecessary for list view that only shows title, state, priority and assignee.
 - No Pagination – Returns all issues in a single response regardless of what the frontend needs. With realistic production data, 1000+ issues, this will degrade linearly.
2. GET /api/dashboard/my-week (68ms P99) – Slowest lightweight endpoint. Root causes:
 - 7 sequential DB queries. The handler runs queries one after another: person lookup -> workspace config -> weekly plan -> etc. Each query is fast but sequential execution means latency stacks additively. At 50 connections, connection pool contention amplifies this.
 - No query batching. Several queries are independent (plan + retro + standups) which could be run in parallel with `Promise.all()` but are instead executed sequentially.

Priority	Endpoint	Issue	Fix	Impact Now	Impact at Scale
1	GET /api/issues	No pagination, 310KB payload with unused `content` field	Remove `content` from list response. Add server-side pagination	~170ms P99 reduction	Grows linearly with issue count
2	GET /api/dashboard/my-week	7 sequential DB queries that could run in parallel	Parallelize independent queries with <i>Promise.all()</i>	~5-8ms P99 reduction	Constant per request

Minor Bottlenecks:

- GET /api/issues (203ms P99):
 - Two sequential DB queries – issues + batch associations. Handler fetches all issues with JOINS (query takes ~1ms) then runs `getBelongsToAssociationsBatch()` to fetch program/sprint/project associations for all 404 issues in the DB via `document_associations`. Both queries are efficient individually, but they run sequentially and the results must be merged in
 - Only 1-2ms P99 reduction for current scale but at larger scales this grows linearly with more programs & sprints.
- AUTH overhead:
 - There is no index on `token_hash`. The Bearer token auth does a SELECT on `api_tokens` which does a full table scan per request.
 - `UPDATE last_used_at` runs on **every single request** – Under 285 RPS, that's 570 extra DB queries/sec just for auth.
 - Missing index on `token_hash` has minor (< 1ms) impact at current scale but scales linearly with token count. Cheap fix that prevents future problems.
 - Throttling `last_used_at` UPDATE from per-request to once a minute has minor (<1ms) impact at all scales. The real benefit is reducing DB write load. It's an operational health improvement vs latency improvement.

Priority	Endpoint	Issue	Fix	Impact Now	Impact at Scale
3	GET /api/programs & GET /api/weeks	Correlated COUNT subqueries (N×M hidden queries per row)	Replace correlated COUNT subqueries with a single JOIN + GROUP BY	~1-2ms reduction	Grows linearly with more programs and sprints added
4	Auth middleware	<code>`UPDATE last_used_at`</code> on every request	Throttle <code>`last_used_at`</code> updates (e.g., once per minute)	< 1ms	Reduces DB write load (operational). No latency impact
5	Auth middleware	no index on <code>`token_hash`</code>	Add index on <code>`api_tokens.token_hash`</code>	< 1ms	Grows linearly with token count

Improvement Target

20% reduction in P95 response time on at least 2 endpoints. You must provide before/after benchmarks run under identical conditions (same data volume, same concurrency, same hardware). Document the root cause of each bottleneck.

Category 4: Database Query Efficiency

What you are measuring: How efficiently the application queries the database. The unified document model (everything in one table) creates specific query patterns worth examining. You are looking for N+1 queries, missing indexes, full table scans, and unnecessary data fetching.

How to Measure

- Enable PostgreSQL query logging (**log_statement = 'all'** in postgresql.conf or via Docker environment variables)
- Execute 5 common user flows: load the main page, view a document, list issues, load a sprint board, search for content
- Count total queries executed per flow
- Run **EXPLAIN ANALYZE** on the slowest queries
- Check for missing indexes by examining WHERE clauses against existing indexes
- Identify N+1 patterns: places where a list view triggers one query per item instead of a batch query

Audit Deliverable

User Flow	Total Queries	Slowest Query (ms)	N+1 Detected?
Load main page	22	1.08ms	No
View a document	11	0.92ms	No
List issues	5	2.76ms	No
Load sprint board	16	1.12ms	No
Search content	5	0.99ms	No

Improvement Target

20% reduction in total query count on at least one user flow, or 50% improvement on the slowest query. Provide before/after **EXPLAIN ANALYZE** output. Document what was inefficient and why your change fixes it.

20% Query count reduction:

Target: Load main page 22 queries -> 14 queries 36% reduction.

11 of 22 queries for *Load main page* flow are auth/permission overhead.

1. Combine token lookup + workspace role check into one query (saves 3)
2. Throttle *last_used_at* UPDATE to once per minute (saves 4)
3. Combine duplicate *SELECT sprint_start_date FROM workspaces* (saves 1)

50% Improvement on slowest Query:

Target: Issues list query 1.55ms -> 0.38ms

1. Remove the **person_doc LEFT JOIN** which checks if the assignee is archived. This is a rare condition that doesn't justify joining the documents table against itself on a JSONB text comparison for every single request.
 - a. Instead of checking if assignee is archived on every "issues list" query, we can instead implement a lazy lookup on single-issue view only. So when a user selects a specific issue we keep the **person_doc JOIN** on GET /api/documents/:id (single issue view) where it's a JOIN for only one row. Remove **person_doc JOIN** from GET /api/issues as listed above which causes a JOIN across ALL rows. The user rarely sees archived assignees in a list – they only notice when they select and open an issue.
2. Remove d.content from the list query's SELECT which fetches the full TipTap document body for all issues that the list view never displays. The response drops to ~38KB — same information the frontend actually uses.
 - a. D.content is the full rich-text document body stored as TipTap JSON. Every issue has one – it's what you see in the editor when you open an issue:

```
{
  "type": "doc",
  "content": [
    {
      "type": "heading",
      "attrs": { "level": 2 },
      "content": [{ "type": "text", "text": "Description" }]
    },
    {
      "type": "paragraph",
      "content": [{ "type": "text", "text": "Steps to reproduce: ..." }]
    },
    {
      "type": "bulletList",
      "content": [...]
    }
  ]
}
```

- b. The issues list endpoint (GET /api/issues) selects d.content for all issues and sends it in the response. But the list view only renders title, state, priority, assignee, and ticket number — it never displays the document body. That content is only needed when you click into an individual issue, which is handled by GET /api/documents/:id
- c. That's why the response is 310KB. Remove d.content from the list query's SELECT and the response drops to ~38KB — same information the frontend actually uses.

Category 5: Test Coverage and Quality

What you are measuring: What the existing test suite covers, what it misses, and how reliable it is. Ship has 73+ Playwright E2E tests. Your job is to understand what they test, find the gaps, and assess test reliability.

How to Measure

- Run the full test suite: **pnpm test**. Record pass/fail counts and total runtime • Read the test files. Catalog what user flows are covered and which are not
 - Identify flaky tests: run the suite 3 times and note any tests that pass sometimes and fail others
- Map critical user flows (document CRUD, real-time sync, auth, sprint management) against existing test coverage
- If code coverage tooling is not configured, configure it and report line/branch coverage per package

Audit Deliverable

Metric	Your Baseline
Total tests	16 web (all broken) + 451 API + 882 E2E (all broken) = 1,333
Pass / Fail / Flaky	API: 451 / 0 / 0 Web: 0 / 16 / 0 E2E: 0 / 882 / 0
Suite runtime	13.38s (API)
Critical flows with zero coverage	ALL frontend (web/) code – zero tests ran for React components, hooks or UI logic. The web tests suite break due to ESM - CJS conflict. E2E flows – the e2e/ Playwright tests did not run as part of the pnpm test. Those run separately and cover UI flows. Not included here. E2E test suite is also broken due to ESM - CJS conflict. Bundle/build pipeline – no tests for Vite config, build output or asset generation. Collaboration/WebSocket – the two collaboration tests that did run collaboration.test.ts, api-content-preservation.test.ts are unit-level. Real-time multi-user sync under concurrent load has no integration test coverage. Authentication edge cases – PIV/CAIA auth paths are skipped entirely (CAIA not configured, skipping initialization appears in every test file). All 28 suites ran without the government auth provider. dashboard.ts – 1.98% statements. The my-work, my-focus, and my-week endpoints that fire on every login are almost completely untested. programs.ts – 5.05% The entire programs feature is effectively untested. weekly-plans.ts – 4.80% Sprint planning workflows have near-zero

	coverage. accountability.ts (route) – 6.25% The government accountability tracking feature has essentially no tests. comments.ts – 8.98% Comments were added in migration 035 and never tested. team.ts – 8.70% Team grid, assignments, and accountability grid endpoints all untested. collaboration/index.ts – 8.35% The entire Yjs WebSocket sync engine has 8% coverage – This is the most critical runtime component in the app. invite-acceptance.ts service – 0% the invite flow has no coverage at all. allocation.ts util – 0% Resource allocation logic is completely untested.
Code coverage % (if measured)	Web: 0% not measured - test is broken because of dependency. api: 40.34 % statements 33.44 % branches

Only 34% of tests (451/1,333) can actually execute. Two ESM/CJS dependency conflicts have silently broken both the web unit tests and the entire E2E suite:

- E2E: `get-port@7.1.0` is ESM-only, fix by pinning to `get-port@6.1.2`
- WEB: `html-encoding-sniffer@6.0.0` -> `@exodus/bytes` ESM conflict, fix by switching to `happy-dom` (or pinning the dependency)

ESM vs CJS:

[node.js](#) has two module systems–

- CJS (CommonJS): The old one. Uses `require()` and `module.exports`
- ESM (ES Modules): The new one. Uses `import` and `export`

The critical rule: CJS code cannot `require()` an ESM package. It throws `ERR_REQUIRE_ESM`. ESM code *can* import CJS packages, but not the other way around.

E2E issue:

- `get-port` finds an available TCP port on your machine. The E2E fixture uses it to assign unique ports to each parallel test worker's API server.
- `get-port@6.x` was CJS (exported via `module.exports`).
- The author released `get-port@7.0.0` as ESM-only (exports via `export default`).
- Playwright compiles TypeScript to CJS, so `import getPort from 'get-port'` becomes `require('get-port')`
- This crashes because v7 is ESM-only
- Pinning to 6.1.2 fixes it. Pinning means locking a dependency to a specific version instead of allowing upgrades. v6.1.2 is the last version to use CJS so `require()` works fine.

WEB issues:

- html-encoding-sniffer -> @exodus/bytes: This means html-encoding-sniffer internally depends on @exodus/bytes. The chain is: Web tests -> jsdom ->html-encoding-sniffer -> @exodus/bytes (ESM-only)
- html-encoding-sniffer@6.0.0 uses require() to load @exodus/bytes, but @exodus/bytes is ESM-only. Same ERR_REQUIRE_ESM crash as E2E. Just one layer deeper in the dependency chain. We didn't install @exodus/bytes directly; it's a transitive dependency.
- jsdom is what simulates browser DOM for web tests. Jsdom is heavy (~20MB) and has a deep dependency tree. That's where html-encoding-sniffer comes from.
- happy-dom: is a lighter alternative to jsdom that does the same thing, simulates browser DOM during testing, but with fewer dependencies and no ESM conflicts. This sidesteps the problem because happy-dom doesn't depend on html-encoding-sniffer at all.
- Switching to happy-dom is a well-known fix for this specific jsdom ESM breakage – common issue in the vitest ecosystem since severely jsdom transitive dependencies have gone ESM-only.
- Pinning html-encoding-sniffer works for now but would require pnpm overrides in package.json which overrides jsdom's own dependency declaration. If jsdom actually needs a feature from a new version, things would break silently. That's why switching to happy-dom is the better way to go.

Improvement Target

Add meaningful tests for 3 previously untested critical paths, or fix 3 flaky tests with documented root cause analysis. "Meaningful" means the test catches a real regression, not just asserting that a page loads. Each test must include a comment explaining what risk it mitigates.

Category 6: Runtime Error and Edge Case Handling

What you are measuring: How the application behaves when things go wrong. This covers error boundaries, unhandled promise rejections, network failure recovery (especially during real-time collaboration), malformed input handling, and user-facing error states.

How to Measure

- Open browser DevTools and monitor the console during normal usage. Count errors and warnings
- Test network failure: disconnect while editing a document collaboratively, then reconnect. Does data survive? Does the UI recover?
- Test malformed input: submit empty forms, extremely long text, special characters, HTML/script injection
- Test concurrent edge cases: two users editing the same document field simultaneously
- Throttle the network to 3G and use the app. Note every spinner that hangs, every silent failure, every missing loading state
- Check server logs for unhandled errors during all of the above

Audit Deliverable

Metric	Your Baseline
Console errors during normal usage	Switching between multiple docs quickly throws 429 too many requests error. Fast typing with cursor movement into a doc quickly causes 429 too many requests error. Random can't establish a connection to server at ... when selecting a doc. Only happened specifically to the doc with id '86e2ddf9-530e-4eb6-ac03-5b697da52da0'
Unhandled promise rejections (server)	index.ts : 1. No <code>process.on('unhandledRejection')</code> handler – an escaped promise rejection terminates the process with no logging 2. No <code>process.on('uncaughtException')</code> handler – synchronous throws outside try/catch crash silently 3. No <code>pool.on('error')</code> handler in the DB client – connection pool errors go undetected 4. No global express error middleware in app.ts – no <code>app.use((err, req, res, next) => ...)</code> registered after routes
Network disconnect recovery	Partial
Missing error boundaries	1. Main.tsx – no root-level <code><ErrorBoundary></code> wrapping the entire app tree. If any provider throws it'll give complete white screen with no recovery. 2. App.tsx – <code>ErrorBoundary</code> wraps <code><Outlet /></code> only. Sidebar, nav, and layout are unprotected. Errors in these sections will crash the whole page. 3. UnifiedDocumentPage.tsx tab components –

	<Suspense> wraps tabs but no <ErrorBoundary> per tab. Tab render error propagates up to the page-level boundary, losing all document context. 4. Context providers (DocumentsContext, ProgramsContext, etc) – No error boundaries around context consumers. A context error in one panel crashes the entire view.
Silent failures identified	1. Tab query errors swallowed (ProgramProjectsTab.tsx, WeekReconciliation.tsx) – Throttle network to offline, open a program document, click Projects tab. Tab shows empty list with no error message; user thinks there are no projects. 2. WebSocket ws.send() failures in broadcastToUser() – Kill a client connection while the server sends broadcast. ws.send() throws, no catch; broadcast silently dropped for that user. 3. Document persistence failure after disconnect – Stop PostgreSQL while users are editing, wait >30 seconds, turn back on. persistDocument() fails, logs to console only; after 30s doc unloads from memory; edits lost permanently. 4. parseInt NaN passed to SQL (admin.ts, search.ts) – call GET /api/admin/audit-logs?limit=abc . parseInt("abc") returns NaN; PostgreSQL receives NaN for LIMIT, query may error or return unexpected results. 5. Title field no maxLength – Paste 10,000 characters into the document title. Frontend accepts it; backend Zod schema truncates/rejects at 255 chars; no client-side feedback. 6. Session extends network blip = forced logout – Throttle to offline right as inactivity warning fires. <i>extend-session</i> fetch fails, catch block forces logout immediately with no retry.

Improvement Target

Fix 3 error handling gaps. At least one must involve a real user-facing data loss or confusion scenario (not just a missing loading spinner). Each fix requires reproduction steps, before/after behavior, and a screenshot or recording.

429 Errors:

Typing fast – WebSocket message rate limit is too aggressive. 1 second window @ max 50 messages per second per connection. Every keystroke in the TipTap editor generates a Yjs update that gets sent as a WebSocket message. Yjs can batch multiple operations per update cycle, and each awareness update (cursor position, selection changes) is a separate message. Fast typing with cursor movements + awareness broadcasts can exceed 50 msg/sec. When the limit is hit, messages are silently dropped (no error sent back). After 50 consecutive violations, the connection is force-closed with code 1008. The client then reconnects and syncs.

Switching documents quickly – Connection rate limit too low. 1 minute window @ max 30 connections per minute per IP. Every document navigation creates 2 new WebSocket connections (/collaboration/{docType}:{docID} & /events). The old connections are cleaned up via wsProvider.destroy() but recordConnectionAttempt() counts every connection **attempt** within the 60-second window. Even if the prior ones were properly closed.

So switching between 16 documents within 60 seconds ($16 \times 2 = 32$ connections) hits the rate limit. The server responds with a raw HTTP/1.1 429 Too Many Requests and destroys the socket - no WebSocket upgrade happens, no helpful error reaches the frontend, and y-websocket just sees a failed connection and retries (generating more attempts).

Symptom	Rate Limit	Threshold	Why It's Too Aggressive
Fast typing → 429	Message rate	50 msg/sec/connection	Yjs updates + awareness changes can burst above this
Document switching → 429	Connection rate	30 conn/min/IP	Each doc = 2 connections; 16 doc switches in 60s triggers it

Both limits are DDoS protections that are calibrated for attack scenarios, not normal single-user usage patterns. The connection limit especially doesn't account for the fact that closed connections should free up capacity.

Category 7: Accessibility Compliance

What you are measuring: Ship claims Section 508 compliance and WCAG 2.1 AA conformance. Your job is to verify those claims. This means automated accessibility scanning, keyboard navigation testing, screen reader testing, and color contrast verification across the application's major pages.

How to Measure

- Run Lighthouse accessibility audits on every major page of the application. Record the score for each
- Run an automated accessibility scanner (**axe-core**, **pa11y**, or the axe browser extension) and categorize violations by severity (Critical, Serious, Moderate, Minor)
- Test full keyboard navigation: can you reach every interactive element using only Tab, Enter, Escape, and arrow keys?
- Test with a screen reader (VoiceOver, NVDA, or similar). Can you understand the page structure and interact with all controls?
- Check color contrast ratios on text, buttons, and interactive elements against the WCAG 2.1 AA 4.5:1 minimum

Audit Deliverable

Metric	Your Baseline		
Lighthouse accessibility score (per page)	my week main	/my-week	96/100
	Docs main view	/docs	100/100
	Docs selected wiki	/docs/:id	100/100
	Selected Program view	/docs/:id	94/100
	Selected Programs - issues view	/docs/:id/issues	95/100
	Selected Programs - projects view	/docs/:id/projects	91/100
	Selected Programs - weeks view	/docs/:id/weeks	91/100
	Selected issues view	/docs/:id	95/100
	Selected Projects view	/docs/:id	95/100
	Selected Projects details view	/docs/:id/details	92/100
	Selected Projects week view	/docs/:id/weeks	91/100
	Selected Projects retro view	/docs/:id/retro	92/100
	Program main view	/programs	100/100
	Projects main view	/projects	96/100
	Team allocation view	/team/allocation	96/100
	Team directory view	/team/directory	100/100
	Selected team member	/team/:id	100/100
	Team status view	/team/status	96/100
	Team reviews	/team/reviews	96/100
	Team org-chart	/team/org-chart	100/100
	Settings page	/settings	95/100
	Settings invites	/settings?tab=invites	94/100

	Settings API tokens Settings audit logs Settings conversions	/settings?tab=tokens /settings?tab=audit /settings/conversions	93/100 100/100 100/100
Total Critical/Serious violations	Critical: - Title placeholder contrast ~1.6:1 (needs 4.5:1) - No root-level <ErrorBoundary> – provider errors cause white screen with no recovery Serious: - Search input in BacklogPickerModal has no aria-label - Search input in MultiAssociationChips has no aria-label - CommandPalette input missing explicit aria-label - EmojiPicker trigger button has no aria-label - Merge dialog placeholder contrast ~3.2:1 - TabBar has no ArrowLeft/ArrowRight keyboard navigation (WCAG tab pattern) - PropertyRow labels not associated with inputs via htmlFor/id - Decorative SVG icons inside button missing aria-hidden="true" throughout BulkActionBar, ApprovalButton, CommandPalette - text-muted/50 used as content text (not just placeholder) in MyWeekPage line numbers – contrast ~3.2:1		
Keyboard navigation completeness	Partial Broken or missing: - TabBar arrow key navigation – User must tab through every tab instead of using arrows - SelectableList rows missing visible focus ring – Focused row has no visual indicator - KanbanBoard cards have focus:outline-none with no replacement – Focused issue cards invisible to keyboard users - DashboardSidebar buttons lack focus styling – No visible focus on sidebar toggle buttons - Combobox dropdowns (Person, Project, Program) arrow key nav unclear – Radix Popover provides some support but no explicit binding visible		
Color contrast failures	- Editor title placeholder – #8a8a8a at 30% opacity → effective ~#2a2a2a. Ratio ~1.6:1 Required 4.5:1 = FAIL - MergeProgramDialog search placeholder – #8a8a8a at 50% opacity → ~#4a4a4a Ratio ~3.2:1 Required 4.5:1 = FAIL - MyWeekPage line numbers (text-muted/50) – #8a8a8a at 50% opacity → ~#4a4a4a Ratio ~3.2:1 Required 4.5:1 = FAIL - WorkspaceSettings/Programs dash (text-muted/50) – #8a8a8a at 50% opacity → ~#4a4a4a Ratio ~3.2:1 Required 4.5:1 = FAIL - Editor body placeholder (#525252) – #525252 on #0d0d0d Ratio ~3.2:1 Required 4.5:1 = FAIL - Disabled buttons (opacity-50 on bg-accent) – #005ea2 at 50% → ~#002f51 Ratio ~1.8:1 Required 4.5:1 = FAIL		
Missing ARIA labels or roles	- BacklogPickerModal.tsx:244 – missing aria-label on search input - MultiAssociationChips.tsx:172 – missing aria-label on search input - CommandPalette.tsx:254 – missing aria-label on search input		

	4. EmojiPicker.tsx:56 – missing aria-label on emoji trigger button 5. PropertyRow.tsx:15 – missing htmlFor / id association between labels and inputs 6. PropertyRow.tsx:21 – missing aria-describedby for required field indicator 7. Multiple icon buttons (BulkActionBar, ApprovalButton, CommandPalette) – missing aria-hidden="true" on decorative SVG icons inside labeled buttons
--	--

Improvement Target

Achieve a Lighthouse accessibility score improvement of 10+ points on the lowest-scoring page, or fix all Critical/Serious violations on the 3 most important pages. Provide before/after Lighthouse reports or axe scan output as evidence.